Раздел 1

1. Войдите под пользователем user1 из практики 2 (su - user1)
2. Подсчитайте количество процессов, имеющих несколько потоков выполнения

```
root@eltex-practice2-pg2-v3:~# su - user1
Warning: your password will expire in 3 days.
user1@eltex-practice2-pg2-v3:~$ ps -eo nlwp --no-headers | awk '$1 > 1' | wc -l
10
user1@eltex-practice2-pg2-v3:~$ top
```

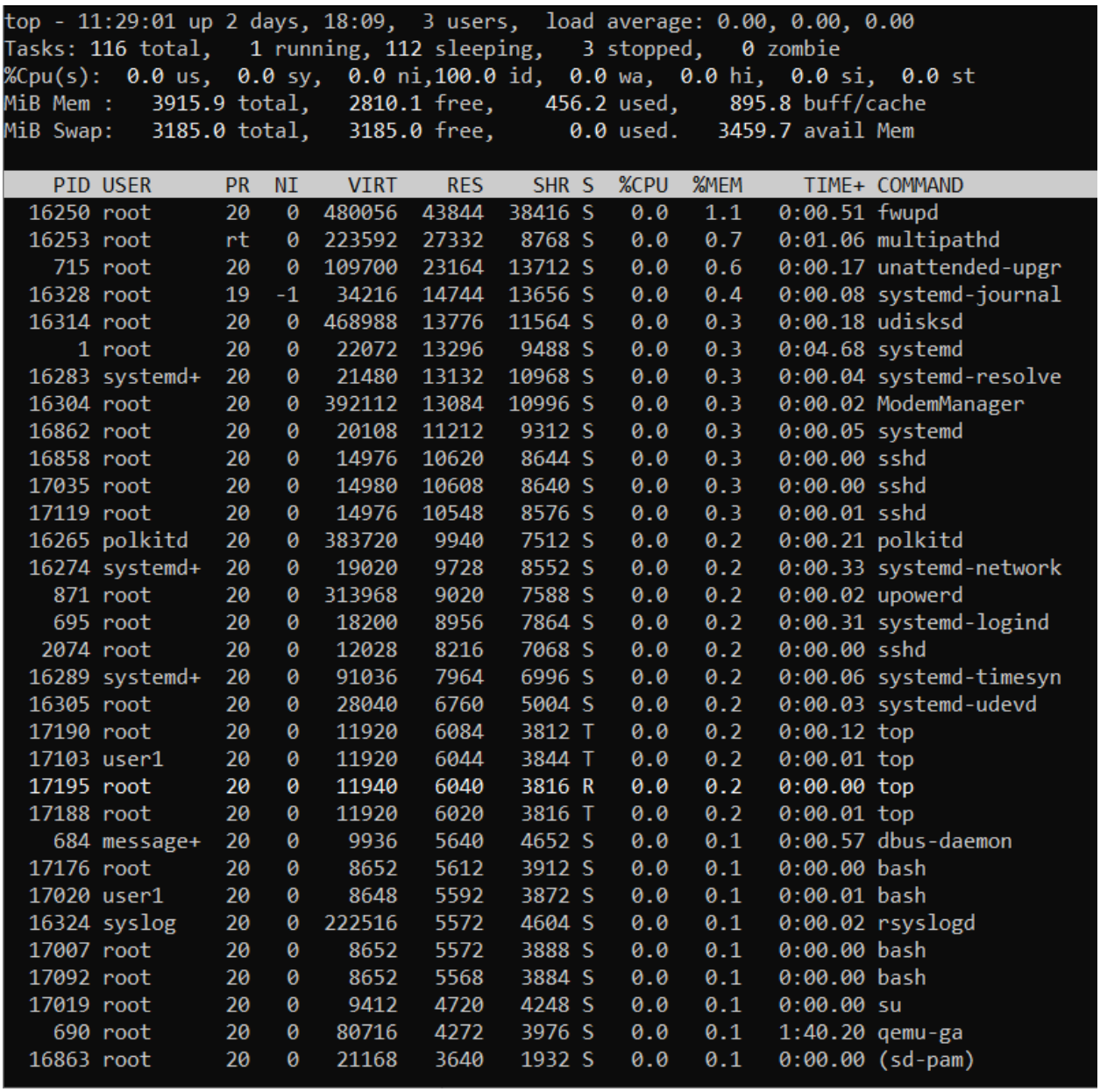
3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:

- отсортируйте список процессов по убыванию значения поля VIRT,

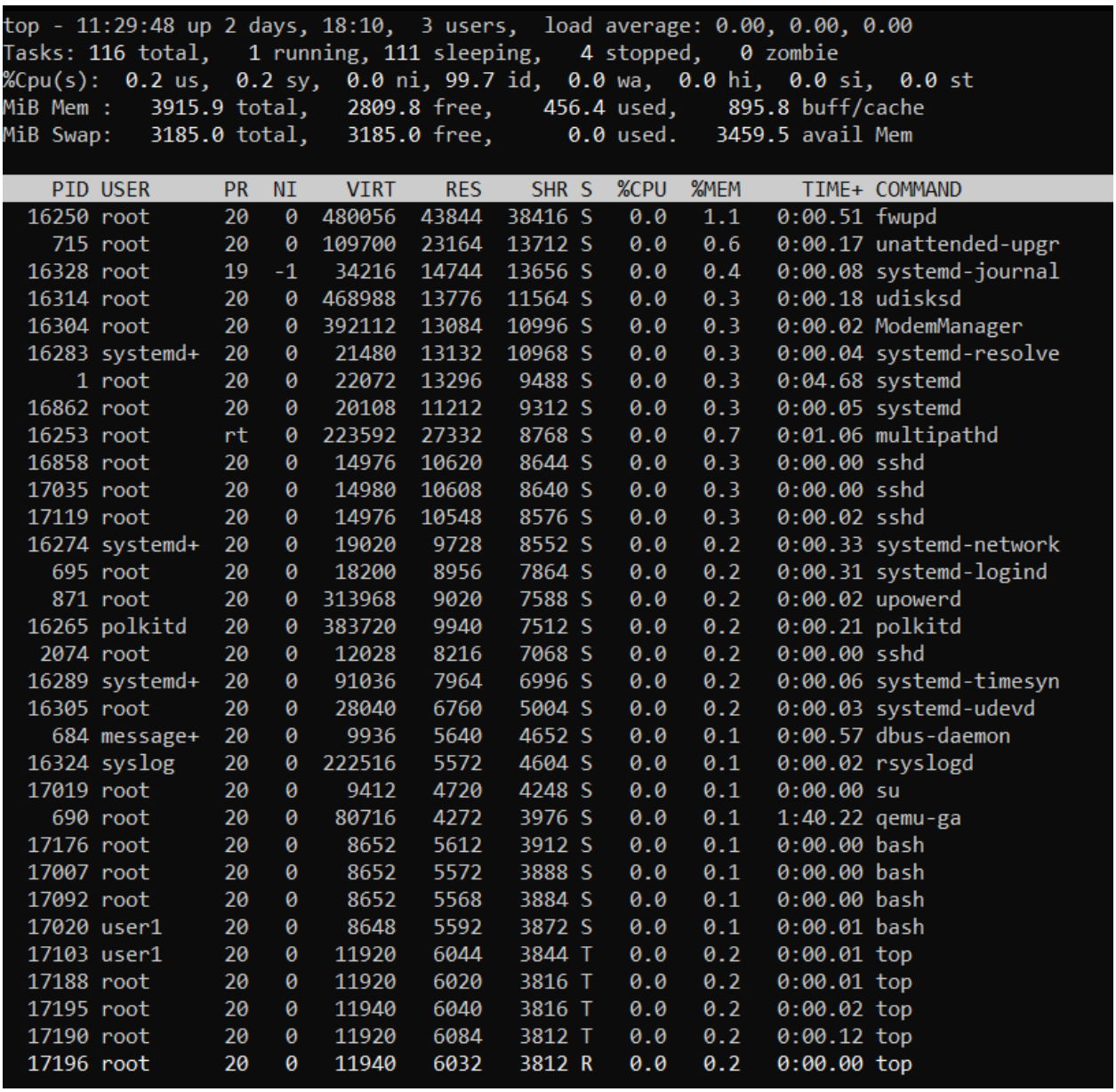
```
top - 11:25:11 up 2 days, 18:06,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 113 sleeping,   2 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2809.8 free,    456.4 used,    895.8 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3459.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  16250 root      20   0  480056  43844  38416 S   0.0   1.1   0:00.50 fwupd
  16314 root      20   0  468988  13776  11564 S   0.0   0.3   0:00.18 udisksd
  16304 root      20   0  392112  13084  10996 S   0.0   0.3   0:00.02 ModemManager
  16265 polkitd   20   0  383720   9940   7512 S   0.0   0.2   0:00.21 polkitd
    871 root      20   0  313968   9020   7588 S   0.0   0.2   0:00.02 upowerd
  16253 root      rt   0  223592  27332   8768 S   0.0   0.7   0:01.04 multipathd
  16324 syslog    20   0  222516   5572   4604 S   0.0   0.1   0:00.02 rsyslogd
    715 root      20   0  109700  23164  13712 S   0.0   0.6   0:00.17 unattended-upgr
  16289 systemd+  20   0   91036   7964   6996 S   0.0   0.2   0:00.05 systemd-timesyn
    690 root      20   0   80716   4272   3976 S   0.0   0.1   1:40.09 qemu-ga
  16328 root      19  -1   34216  14744  13656 S   0.0   0.4   0:00.08 systemd-journal
  16305 root      20   0   28040   6760   5004 S   0.0   0.2   0:00.03 systemd-udevd
      1 root      20   0   22072  13296   9488 S   0.0   0.3   0:04.67 systemd
  16283 systemd+  20   0   21480  13132  10968 S   0.0   0.3   0:00.04 systemd-resolve
  16863 root      20   0   21168   3640   1932 S   0.0   0.1   0:00.00 (sd-pam)
  16862 root      20   0   20108  11212   9312 S   0.0   0.3   0:00.05 systemd
  16274 systemd+  20   0   19020   9728   8552 S   0.0   0.2   0:00.33 systemd-network
    695 root      20   0   18200   8956   7864 S   0.0   0.2   0:00.31 systemd-logind
  17035 root      20   0   14980  10608   8640 S   0.0   0.3   0:00.00 sshd
  16858 root      20   0   14976  10620   8644 S   0.0   0.3   0:00.00 sshd
  17119 root      20   0   14976  10548   8576 S   0.0   0.3   0:00.01 sshd
   2074 root      20   0   12028   8216   7068 S   0.0   0.2   0:00.00 sshd
  17103 user1     20   0   11920   6044   3844 T   0.0   0.2   0:00.01 top
```

- *Virt (Virtual Image) – виртуальная память (зарезервированная, но физически ещё не использованная)*
- зафиксируйте результат скриншотом, повторите для полей RES и SHR, в отчете поясните, что описывают данные показатели;



- *Res (Resident Size) - Резидентная память (сколько физически памяти занимает процесс)*



- *SHR (Shared Memory) – разделяемая память (физическая память, которую процесс делит с другими процессами)*

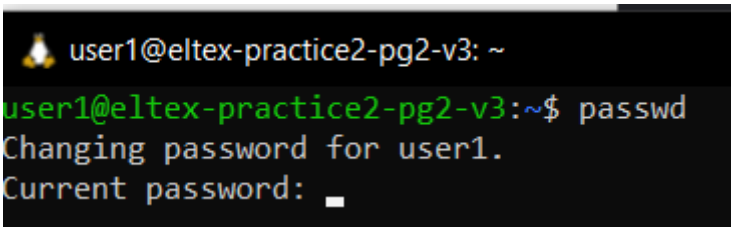
- удалите поля VIRT, RES, SHR и добавьте поле RUSER, сделайте так, чтобы это поле было показано после поля USER, поясните отличие RUSER/USER;

```
top - 11:48:52 up 2 days, 18:29,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 118 total,   1 running, 112 sleeping,   5 stopped,   0 zombie
%Cpu(s):   0.0/0.0     0[
MiB Mem :   3915.9 total,   2806.4 free,    459.7 used,    895.9 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3456.2 avail Mem

    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
  17020 user1     user1     20   0 S   0.0   0.1   0:00.01 bash
  17103 user1     user1     20   0 T   0.0   0.2   0:00.01 top
  16289 systemd+  systemd+  20   0 S   0.0   0.2   0:00.06 systemd-timesyn
  16283 systemd+  systemd+  20   0 S   0.0   0.3   0:00.05 systemd-resolve
  16274 systemd+  systemd+  20   0 S   0.0   0.2   0:00.35 systemd-network
  16324 syslog    syslog    20   0 S   0.0   0.1   0:00.02 rsyslogd
      1 root      root      20   0 S   0.0   0.3   0:04.69 systemd
      2 root      root      20   0 S   0.0   0.0   0:00.05 kthreadd
      3 root      root      20   0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-netns
     12 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      root      20   0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      root      20   0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      root      20   0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      root      20   0 S   0.0   0.0   0:00.06 ksoftirqd/0
     17 root      root      20   0 I   0.0   0.0   0:01.88 rcu_preempt
     18 root      root      rt   0 S   0.0   0.0   0:00.85 migration/0
     19 root      root     -51   0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      root      20   0 S   0.0   0.0   0:00.00 cpuhp/0
     21 root      root      20   0 S   0.0   0.0   0:00.00 cpuhp/1
     22 root      root     -51   0 S   0.0   0.0   0:00.00 idle_inject/1
     23 root      root      rt   0 S   0.0   0.0   0:00.82 migration/1
     24 root      root      20   0 S   0.0   0.0   0:00.06 ksoftirqd/1
     26 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     27 root      root      20   0 S   0.0   0.0   0:00.00 kdevtmpfs
     28 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-inet_
     29 root      root      20   0 S   0.0   0.0   0:00.00 kauditd
     30 root      root      20   0 S   0.0   0.0   0:00.04 khungtaskd
     32 root      root      20   0 S   0.0   0.0   0:00.00 oom_reaper
```

- *RUSER - Real User Name (Имя реального владельца процесса, который физически запустил процесс). USER - учетная запись, права которой процесс использует в данный момент*

4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля



5. Перейдите в терминальное окно с top и выполните следующие действия:

- выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;

```
top - 12:14:01 up 2 days, 18:54,  4 users,  load average: 0.00, 0.00, 0.00
Tasks: 135 total,   1 running, 121 sleeping,  13 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 95.2 id,  0.0 wa,  0.0 hi,  4.8 si,  0.0 st
MiB Mem :   3915.9 total,   2758.2 free,    487.2 used,    916.8 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3428.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  17020 user1     20   0    8648   5592   3872 S   0.0   0.1   0:00.01 bash
  17103 user1     20   0   11920   6044   3844 T   0.0   0.2   0:00.01 top
  17310 user1     20   0    8648   5600   3884 S   0.0   0.1   0:00.02 bash
  17321 user1     20   0   11940   6016   3800 T   0.0   0.2   0:00.00 top
  17324 user1     20   0   11944   6040   3816 T   0.0   0.2   0:00.04 top
  17339 user1     20   0   11944   6048   3816 T   0.0   0.2   0:00.05 top
  17344 user1     20   0   11944   6040   3816 T   0.0   0.2   0:00.04 top
  17354 user1     20   0   11944   6024   3800 T   0.0   0.2   0:00.00 top
  17362 user1     20   0    8648   5580   3872 S   0.0   0.1   0:00.01 bash
  17381 user1     20   0   11944   6040   3816 T   0.0   0.2   0:00.48 top
  17394 user1     20   0   11944   6040   3816 T   0.0   0.2   0:00.11 top
  17397 user1     20   0   11944   5784   3560 R   0.0   0.1   0:00.00 top
```

- найдите процесс, запущенный командой passwd;

```
user1@eltex-practice2-pg2-v3:~$ ps -a
    PID TTY          TIME CMD
  17019 pts/0    00:00:00 su
  17020 pts/0    00:00:00 bash
  17103 pts/0    00:00:00 top
  17188 pts/2    00:00:00 top
  17190 pts/2    00:00:00 top
  17195 pts/2    00:00:00 top
  17196 pts/2    00:00:00 top
  17222 pts/2    00:00:00 top
  17309 pts/2    00:00:00 su
  17310 pts/2    00:00:00 bash
  17321 pts/2    00:00:00 top
  17324 pts/2    00:00:00 top
  17339 pts/2    00:00:00 top
  17344 pts/2    00:00:00 top
  17354 pts/2    00:00:00 top
  17361 pts/3    00:00:00 su
  17362 pts/3    00:00:00 bash
  17381 pts/2    00:00:00 top
  17392 pts/3    00:00:00 passwd
  17394 pts/2    00:00:00 top
  17395 pts/2    00:00:00 ps
```

- *Поскольку passwd выполняется от root2*
- отправьте этому процессу сигналы SIGTERM, SIGINT, SIGQUIT, SIGKILL используя их числовые идентификаторы (man 7 signal)

```
user1@eltex-practice2-pg2-v3:~$ passwd
Changing password for user1.
Current password: Killed
user1@eltex-practice2-pg2-v3:~$ passwd
Changing password for user1.
Current password: Killed
user1@eltex-practice2-pg2-v3:~$
```

- *Процесс завершился после SIGQUIT (pid 17392) и после SIGKILL (pid 17404)*

6. Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z
7. Выполните команду sleep 600, нажмите Ctrl-Z и выполните команду jobs

```
user1@eltex-practice2-pg2-v3:~$ vim ~/file_task3.txt

[11]+  Stopped                 vim ~/file_task3.txt
user1@eltex-practice2-pg2-v3:~$ sleep 600
^Z
[12]+  Stopped                 sleep 600
user1@eltex-practice2-pg2-v3:~$ jobs
[1]   Stopped                 top
[2]   Stopped                 top -u user1
[3]   Stopped                 top -u user1
[4]   Stopped                 top -u user1
[5]   Stopped                 top -u user1
[6]   Stopped                 top -u user1
[7]   Stopped                 top -u user1
[8]   Stopped                 top -u user1
[9]   Stopped                 top -u user1
[10]   Stopped                 top -u user1
[11]-  Stopped                 vim ~/file_task3.txt
[12]+  Stopped                 sleep 600
user1@eltex-practice2-pg2-v3:~$
```

8. Последнее задание (sleep 600) сделайте фоновым

```
root@eltex-practice2-pg2-v3:~# jobs
[1]-  Stopped                 sleep 600
[2]+  Stopped                 vim ~/file_task3.txt
root@eltex-practice2-pg2-v3:~# bg %1
[1]- sleep 600 &
```

```
root@eltex-practice2-pg2-v3:~# jobs
[1]-  Running                 sleep 600 &
[2]+  Stopped                 vim ~/file_task3.txt
```

9. Измените число NICE у задания (sleep 600), сделав его равным 10
10. Проверьте, что число NICE у этого задания изменилось

```
root@eltex-practice2-pg2-v3:~# ps
    PID TTY          TIME CMD
  17912 pts/0    00:00:00 bash
  17925 pts/0    00:00:00 sleep
  17934 pts/0    00:00:00 vim
  17942 pts/0    00:00:00 ps
root@eltex-practice2-pg2-v3:~# renice -n 10 -p 17925
17925 (process ID) old priority 0, new priority 10
root@eltex-practice2-pg2-v3:~# ps -l
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0   17912   17796  0  80   0 -  2196 do_wai pts/0    00:00:00 bash
0 S     0   17925   17912  0  90  10 -  1422 do_sys pts/0    00:00:00 sleep
0 T     0   17934   17912  0  80   0 -  6200 do_sig pts/0    00:00:00 vim
4 R     0   17944   17912  0  80   0 -  2731 -      pts/0    00:00:00 ps
root@eltex-practice2-pg2-v3:~#
```

11. Сделайте задание vim ~/file_ task3.txt активным и выйдите из редактора

```
root@eltex-practice2-pg2-v3:~# fg %3
vim ~/file_task3.txt
root@eltex-practice2-pg2-v3:~# jobs
[1]+  Running                 sleep 600 &
root@eltex-practice2-pg2-v3:~#
```

12. Отправьте сигнал SIGTERM заданию sleep 600 и выполните команду jobs

```
root@eltex-practice2-pg2-v3:~# kill -15 %1
root@eltex-practice2-pg2-v3:~# jobs
[1]+  Terminated              sleep 600
root@eltex-practice2-pg2-v3:~#
```

13. Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду trap) и отправьте сигналы самому себе

```
root@eltex-practice2-pg2-v3:~# trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
root@eltex-practice2-pg2-v3:~# kill -SIGINT $$
Меня голыми руками не возьмёшь!
root@eltex-practice2-pg2-v3:~# kill -SIGQUIT $$
Меня голыми руками не возьмёшь!
```

14. Создайте несколько зомби-процессов несколько раз выполнив команду:

(sleep 1 & exec /bin/sleep 30) &

30 – настраиваемая задержка времени существования зомби-процесса в секундах, если не успеваете выполнить следующий пункт – увеличьте время

```
root@eltex-practice2-pg2-v3:~# (sleep 1 & exec /bin/sleep 300) &
[1] 17961
root@eltex-practice2-pg2-v3:~# (sleep 1 & exec /bin/sleep 300) &
[2] 17963
root@eltex-practice2-pg2-v3:~# (sleep 1 & exec /bin/sleep 300) &
[3] 17965
root@eltex-practice2-pg2-v3:~# (sleep 1 & exec /bin/sleep 300) &
[4] 17967
root@eltex-practice2-pg2-v3:~# ps
    PID TTY          TIME CMD
  17912 pts/0    00:00:00 bash
  17961 pts/0    00:00:00 sleep
  17962 pts/0    00:00:00 sleep
  17963 pts/0    00:00:00 sleep
  17964 pts/0    00:00:00 sleep
  17965 pts/0    00:00:00 sleep
  17966 pts/0    00:00:00 sleep
  17967 pts/0    00:00:00 sleep
  17968 pts/0    00:00:00 sleep
  17969 pts/0    00:00:00 ps
```

15. Найдите при помощи команды ps все зомби-процессы, выведите их PID и PPID. Попробуйте досрочно завершить выполнение зомби-процессов.

```
root@eltex-practice2-pg2-v3:~# ps -eo pid,ppid,state,cmd | awk '$3 == "Z" {print "PID: " $1 ", PPID: " $2 " [" $4 "]"}'
PID: 17962, PPID: 17961 [[sleep]]
PID: 17964, PPID: 17963 [[sleep]]
PID: 17966, PPID: 17965 [[sleep]]
PID: 17968, PPID: 17967 [[sleep]]
root@eltex-practice2-pg2-v3:~#
```

```
root@eltex-practice2-pg2-v3:~# ps
    PID TTY          TIME CMD
  17912 pts/0    00:00:00 bash
  17963 pts/0    00:00:00 sleep
  17964 pts/0    00:00:00 sleep
  17965 pts/0    00:00:00 sleep
  17966 pts/0    00:00:00 sleep
  17967 pts/0    00:00:00 sleep
  17968 pts/0    00:00:00 sleep
  17984 pts/0    00:00:00 ps
[1]   Terminated              ( sleep 1 & exec /bin/sleep 300 )
root@eltex-practice2-pg2-v3:~# kill 9 17964
-bash: kill: (9) - No such process
-bash: kill: (17964) - No such process
[2]   Done                    ( sleep 1 & exec /bin/sleep 300 )
root@eltex-practice2-pg2-v3:~# kill 9 17963
-bash: kill: (9) - No such process
-bash: kill: (17963) - No such process
[3]-  Done                    ( sleep 1 & exec /bin/sleep 300 )
[4]+  Done                    ( sleep 1 & exec /bin/sleep 300 )
root@eltex-practice2-pg2-v3:~# kill 15 17963
-bash: kill: (17963) - No such process
root@eltex-practice2-pg2-v3:~#
```

```
root@eltex-practice2-pg2-v3:~# ps
    PID TTY          TIME CMD
  17912 pts/0    00:00:00 bash
  17987 pts/0    00:00:00 sleep
  17988 pts/0    00:00:00 sleep
  17989 pts/0    00:00:00 sleep
  17990 pts/0    00:00:00 sleep
  17991 pts/0    00:00:00 sleep
  17992 pts/0    00:00:00 sleep
  17993 pts/0    00:00:00 ps
```

```
root@eltex-practice2-pg2-v3:~# kill 9 17987
-bash: kill: (9) - No such process
root@eltex-practice2-pg2-v3:~# kill 9 17988
-bash: kill: (9) - No such process
-bash: kill: (17988) - No such process
[1]   Terminated              ( sleep 1 & exec /bin/sleep 300 )
root@eltex-practice2-pg2-v3:~# ps
    PID TTY          TIME CMD
  17912 pts/0    00:00:00 bash
  17989 pts/0    00:00:00 sleep
  17990 pts/0    00:00:00 sleep
  17991 pts/0    00:00:00 sleep
  17992 pts/0    00:00:00 sleep
  17994 pts/0    00:00:00 ps
root@eltex-practice2-pg2-v3:~#
```

Все, что узнали о зомби-процессах и их особенностях в процессе выполнения пунктов опишите, используя ~10 предложений в отчете.

По сути, командой (sleep 1 & exec /bin/sleep 30) & мы создаем родительский процесс и дочерний, но по завершению родительского процесса, дочерний не заканчивает свою работу. Родительский процесс ждет пока завершится дочерний и оба процесса завершают работу. Зомби процесс (дочерний) нельзя завершить через команду kill, скорее всего потому, что уже завершился родительский процесс и обратиться к дочернему напрямую уже нельзя. Дочерний процесс наследует pid родительского, но имеет ppid для обращения из родительского, поэтому нельзя его завершить «из вне». Завершить зомби-процесс можно только принудительно убив родительский.

Раздел 2

1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:
    - Создает именованный канал с именем, указанном с абсолютным путем в параметре запуска скрипта --pishitesuda=
    - Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
    - В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?" возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
    - При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и

завершает свою работу, предварительно удалив именованный канал.

```
root@eltex-practice2-pg2-v3:~# cat cuckoo.sh
#!/bin/bash

FIFO_NAME=""

for arg in "$@"; do
    if [[ "$arg" =~ ^--pishitesuda=(.*)$ ]]; then
        FIFO_NAME="${BASH_REMATCH[1]}"
    fi
done

[ -p "$FIFO_NAME" ] || mkfifo "$FIFO_NAME"
DATE_TIME="$(date) $(date +%T)"
echo "$DATE_TIME Startup!" >> cuckoo.log

exec 3<"$FIFO_NAME"
exec 4>"$FIFO_NAME"

while true; do
    if read -r line <&3; then
        DATE_TIME="$(date) $(date +%T)"
        if [[ "$line" == "poka" ]]; then
                echo "$DATE_TIME Shutdown!" >> cuckoo.log
                break
        fi
        if [[ "$line" =~ ^[^[]+\[[0-9]+\]:\ how\ much\ time\ do\ I\ have\?$ ]]; then
                RAND_NUM=$((RANDOM % 9 + 2))
                echo "$line" | awk -F']' -v randd="$RAND_NUM" '{print $1 "] " randd }' >> cuckoo.log
                # ИСПРАВЛЕНО: пишем в дескриптор 4 и отправляем в фон, чтобы не зависало
                echo "$RAND_NUM" > "$FIFO_NAME"
                sleep 2
        fi
    fi
done

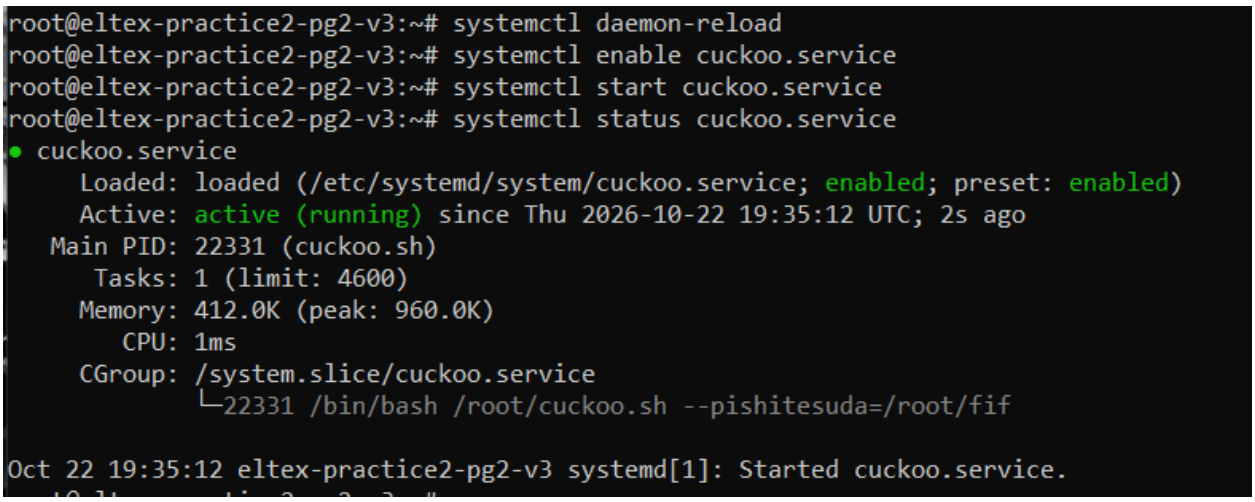
exec 3>&-
exec 4>&-
rm -f "$FIFO_NAME"
root@eltex-practice2-pg2-v3:~#
```

2. Создайте службу systemd (unit) для запуска скрипта cuckoo.sh, разрешите его работу (enable), запустите его и проверьте состояние (status).

```
root@eltex-practice2-pg2-v3:~# cat /etc/systemd/system/cuckoo.service
[Unit]

[Service]
Type=simple
ExecStart=/root/cuckoo.sh --pishitesuda=/root/fif

[Install]
WantedBy=multi-user.target
```



3. Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:

    - При запуске проверят, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
    - При запуске дописывает в файл report_**имя_скрипта_без_полного_пути**.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
    - Запрашивает через именованный канал скрипта cuckoo.sh, который получает из procfs, время сообщением в указанном ранее формате и ждет указанное количество секунд.
    - Дописывает в файл report_**имя_скрипта_без_полного_пути**.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.

```
root@eltex-practice2-pg2-v3:~# cat template_task.sh
#!/bin/bash

if [[ "$0" == "template_task.sh" ]]; then
        echo "я бригадир, сам не работаю"
        exit 0
fi

FILE_NAME="report_$(echo "$0" | awk -F'/' '{print $NF}').log"
DATE_TIME="$(date) $(date +%T)"
RAND_PID=$((RANDOM % 10000 + 10000))
echo "$DATE_TIME [$RAND_PID] Скрипт запущен" >> "$FILE_NAME"

CUCKOO_PID=$(pgrep -f "cuckoo.sh")
CUCKOO_CMD=$(cat -v "/proc/$CUCKOO_PID/cmdline")

if [[ "$CUCKOO_CMD" =~ --pishitesuda=([^^-]+) ]]; then
    FIFO_NAME=$(echo "${BASH_REMATCH[1]}" | tr -d '^@' | sed 's/\^M//g' | xargs)
else
    echo "Ошибка: Путь к FIFO не найден в procfs"
    exit 1
fi

echo "$0[$RAND_PID]: how much time do I have?" > "$FIFO_NAME"
sleep 1
read -r NUM < "$FIFO_NAME"
sleep "$NUM"

DATE_TIME="$(date) $(date +%T)"
echo "$DATE_TIME [$RAND_PID] Скрипт завершился, работал $NUM секунд" >> "$FILE_NAME"
root@eltex-practice2-pg2-v3:~#
```

4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log

```
root@eltex-practice2-pg2-v3:~# cat observer.sh
#!/bin/bash
cd /root

while IFS= read -r line; do
        SCRIPT_PID=$(pgrep -f "$line" | grep -v "$$")
        if [ -z "$SCRIPT_PID" ]; then
                nohup "$line" > /dev/null 2>&1 &
                DATE_TIME="$(date) $(date +%T)"
                echo "$DATE_TIME Скрипт перезапущен: $line" >> /root/observer.log
        fi
done < "/root/observer.conf"
```

5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```
  GNU nano 7.2                                  /tmp/crontab.JyNUqd/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
* * * * * /root/observer.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
```

```
crontab: installing new crontab
```

6. Создайте несколько символьных ссылок на файл template_task.sh с различными именами (рабочие задачи), добавьте в файл конфигурации observer.conf соответствующие записи об этих задачах, включая исходный файл template_task.sh

```
root@eltex-practice2-pg2-v3:~# ln -s template_task.sh link_first.sh
root@eltex-practice2-pg2-v3:~# ln -s template_task.sh link_second.sh
root@eltex-practice2-pg2-v3:~# ln -s template_task.sh link_tretiu.sh
```

```
root@eltex-practice2-pg2-v3:~# cat observer.conf
/root/link_first.sh
/root/link_second.sh
/root/link_tretiu.sh
```

7. Соберите статистику работы в виде набора файлов report_*.log, observer.log, cuckoo.log приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива tar. Не забудьте остановить процессы, удалив задачу в cron и остановив и запретив службу!